

Customizing `std::expected`'s exception

Document #: P3014R0
Date: 2023-10-14
Project: Programming Language C++
Audience: LEWG
Reply-to: Jonathan Müller (think-cell)
<foonathan@jonathanmueller.dev>

Contents

- [1 Abstract](#)
- [2 Motivation](#)
- [3 Prior Art](#)
- [4 Proposed design](#)
- [5 Alternative design](#)
- [6 Open questions](#)
- [7 Acknowledgments](#)
- [8 References](#)

§ 1 Abstract

We propose a way to customize the exception thrown by `std::expected::value()`. That way, we can make `std::expected` more usable in interfaces that want to support both error codes and exceptions.

§ 2 Motivation

[[P0260R7](#)] proposes concurrent queues to the C++ standard library. Their operations can fail, so the current design proposes the `std::filesystem` approach of having two overloads: one that throws, and one that fills a `std::error_code` parameter. [[P2921R0](#)] explores different designs, in particular an approach that uses `std::expected`:

API

Status Quo	<code>std::expected</code>
------------	----------------------------

Status Quo	std::expected
<pre>void push(const T&); bool push(const T&, error_code& ec);</pre>	<pre>auto push(const T&) -> expected<void, conqueue_errc>;</pre>

non-throwing

Status Quo	std::expected
<pre>std::error_code ec; if (q.push(5, ec)) return; println("got {}", ec);</pre>	<pre>if (auto result = q.push(5)) return; else println("got {}", result.error());</pre>

throwing

Status Quo	std::expected
<pre>q.push(5); ... catch(const conqueue_error& e)</pre>	<pre>// Awkward use. q.push(5).or_else([](auto code) { throw conqueue_error(code); }); ... catch(const conqueue_error& e) // Awkward exception type. q.push(5).value(); ... catch(const bad_expected_access<conqueue_errc>& e)</pre>

We propose a way to customize the exception thrown by std::expected::value() using a new std::expected_traits mechanism. When applied to conqueue_errc, it can result in the following interface.

throwing

Status Quo	std::expected, our proposal
<pre>q.push(5); ... catch(const conqueue_error& e)</pre>	<pre>q.push(5).value(); ... catch(const conqueue_error& e)</pre>

§ 3 Prior Art

The `std::expected` paper [P0323R12] has a discussion on this in section 3.16, but it was not proposed and has not been seriously discussed in the committee at the time.

§ 4 Proposed design

The new `std::expected_traits` has a default specialization that throws `std::bad_expected_access<E>`:

```
template <typename E>
struct expected_traits
{
    [[noreturn]] static void throw_error(E e)
    {
        throw std::bad_expected_access<E>(std::move(e));
    }
};
```

Code in `std::expected` that currently throws `std::bad_expected_access` unconditionally (e.g. `.value()`), instead calls `std::expected_traits::throw_error(error())`. As no `std::expected_traits` specialization exists currently, this is not a breaking change.

User code is allowed to customize `std::expected_traits` for their own error types, where `throw_error()` can do whatever is appropriate, as long as it does not return. Crucially, it does not necessarily need to throw, but could also `std::abort()` the program instead.

§ 5 Alternative design

Alternatively, we could introduce a marker type, let's call it `std::unexpected_exception<E, Exception>` and bikeshed later. A `std::expected<T, std::unexpected_exception<E, Exception>>` behaves just like a `std::expected<T, E>`, but instead of throwing `std::bad_expected_access(error())` it throws `Exception(error())`.

That way, the exception associated with an error type can be customized on a per-instance basis instead of globally per error type.

§ 6 Open questions

Should `std::expected_traits` take `<T, E>` and not `<E>`?

This could enable a customization based on specific value-error combinations only.

Should the trait be tied to `std::expected`?

The concept of “exception associated to an error type” seems more general than `std::expected`. We could add it as something more general, like `std::default_error_exception` or `std::error_traits`.

Should the trait be specialized for `std::error_code` to throw `std::system_error`?

It would make a lot of sense, but is unfortunately a breaking change. It would work with the alternative design by using `std::expected<T, std::expected_error<std::error_code, std::system_error>>`.

Should we also customize `.error()`?

Currently it is UB if there is no stored error. It could instead call something on the traits, so a user can customize it to return an error value that means “ok”.

Should we add something like `std::expected::check()` as well?

Calling `.value()` on a `std::expected<void, E>` as in the example is a bit awkward. Maybe we should have `.check()` which does nothing if the expected has a value, but calls `std::expected_traits<E>::throw_error` if it has an error.

§ 7 Acknowledgments

Thanks to JF Bastien and Jonathan Wakely for providing feedback on an initial draft of this paper.

§ 8 References

[P0260R7] Detlef Vollmann, Lawrence Crowl, Chris Mysisen, Gor Nishanov. 2023-07-13. C++ Concurrent Queues.

<https://wg21.link/p0260r7>

[P0323R12] Vicente Botet, JF Bastien, Jonathan Wakely. 2022-01-07. `std::expected`.

<https://wg21.link/p0323r12>

[P2921R0] Gor Nishanov, Detlef Vollmann. 2023-07-05. Exploring `std::expected` based API alternatives for `buffer_queue`.

<https://wg21.link/p2921r0>